

Task of the Day :-

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
from sklearn.preprocessing import LabelEncoder
```

```
np.random.seed(42)
n = 200
locations = ['Urban', 'Suburban', 'Rural']
df = pd.DataFrame({
    'area_sqft': np.random.randint(500, 4000, n),
    'num_rooms': np.random.randint(1, 7, n),
    'location': np.random.choice(locations, n),
    'age_years': np.random.randint(0, 40, n),
})
```

```
loc_map = {'Urban': 1.5, 'Suburban': 1.0, 'Rural': 0.7} # expensive medium cheap
noise = np.random.normal(0, 20000, n)
df['price'] = (
    df['area_sqft'] * 150 +
    df['num_rooms'] * 25000 -
    df['age_years'] * 2000 +
    df['location'].map(loc_map) * 50000 +
    noise
).astype(int)
print(df.head(8).to_string(index=False))
print(f"\nDataset shape : {df.shape}")
print(df.describe()[['area_sqft', 'num_rooms', 'age_years', 'price']].round(0))
```

area_sqft	num_rooms	location	age_years	price
3674	4	Rural	20	641203
1360	4	Urban	35	286449
1794	6	Urban	9	467157
1630	6	Rural	36	345778
1595	3	Urban	8	371644
3592	2	Rural	23	599665
2138	4	Suburban	34	376463
2669	1	Urban	34	435235

```
Dataset shape : (200, 5)
```

	area_sqft	num_rooms	age_years	price
count	200.0	200.0	200.0	200.0
mean	2334.0	3.0	20.0	448863.0
std	992.0	2.0	11.0	154152.0
min	521.0	1.0	0.0	109237.0
25%	1519.0	2.0	10.0	325261.0
50%	2372.0	3.0	20.0	459337.0
75%	3230.0	5.0	30.0	582491.0
max	3999.0	6.0	39.0	779375.0

```
le=LabelEncoder()
df['location_enc']=le.fit_transform(df['location'])
features = ['area_sqft', 'num_rooms', 'age_years', 'location_enc']
X = df[features]
y = df['price']
print(df[['location', 'location_enc']].drop_duplicates().sort_values('location_enc'))
print(f"\nFeature matrix shape : {X.shape}")
print(f"Target vector shape : {y.shape}")
```

	location	location_enc
0	Rural	0
6	Suburban	1
1	Urban	2

```
Feature matrix shape : (200, 4)
Target vector shape : (200,)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(f"Training samples : {len(X_train)} ({len(X_train)/len(X)*100:.0f}%)")
print(f"Testing samples : {len(X_test)} ({len(X_test)/len(X)*100:.0f}%)")
print(f"Feature names : {features}")
```

```

Training samples : 160 (80%)
Testing samples : 40 (20%)
Feature names : ['area_sqft', 'num_rooms', 'age_years', 'location_enc']

```

```

model = LinearRegression()
model.fit(X_train, y_train)
print("Learned Coefficients:")
print("-" * 45)
for feat, coef in zip(features, model.coef_):
    direction = "↑" if coef > 0 else "↓"
    print(f" {feat:15s}: Rs {coef:>10,.0f} {direction}")
print(f" {'intercept':15s}: Rs {model.intercept_:>10,.0f}")
print("-" * 45)

```

Learned Coefficients:

```

-----
area_sqft      : Rs      149 ↑
num_rooms      : Rs    24,007 ↑
age_years      : Rs     -2,089 ↓
location_enc   : Rs    16,852 ↑
intercept      : Rs    41,896
-----

```

```

y_pred = model.predict(X_test)
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print("=" * 40)
print(" MODEL EVALUATION REPORT")
print("=" * 40)
print(f"R² Score : {r2:.4f} {'✓ Excellent' if r2>0.9 else '✓ Good' if r2>0.8 else '■ Needs work'}")
print(f"MAE : Rs {mae:>10,.0f}")
print(f"RMSE : Rs {rmse:>10,.0f}")
print("=" * 40)
print("\nSample Predictions:")
print(f"{'Actual':>10} | {'Predicted':>10} | {'Error':>10}")
print("-" * 36)
for a, p in zip(y_test[:8], y_pred[:8]):
    print(f"{a:>10,} | {p:>10,.0f} | {p-a:>+10,.0f}")

```

```

=====
MODEL EVALUATION REPORT
=====
R² Score : 0.9806 ✓ Excellent
MAE : Rs      17,562
RMSE : Rs     21,467
=====

```

Sample Predictions:

Actual	Predicted	Error
479,483	471,125	-8,358
544,859	572,281	+27,422
669,988	641,023	-28,965
293,622	274,688	-18,934
367,445	341,834	-25,611
150,728	131,297	-19,431
444,343	442,829	-1,514
553,800	548,027	-5,773

```

new_houses = pd.DataFrame({
    'area_sqft': [120, 250, 300, 350],
    'num_rooms': [2, 4, 3, 6 ],
    'age_years': [10, 19, 25, 20 ],
    'location_enc': [2, 1, 0, 2 ],
    'location_lbl': ['Urban', 'Suburban', 'Rural', 'Urban'],
})
predictions = model.predict(new_houses[features])
print("\n HOUSE PRICE PREDICTIONS")
print("=" * 60)
for i, (_, row) in enumerate(new_houses.iterrows()):
    print(f" House {i+1}: {row.area_sqft} sqft | "
          f"{row.num_rooms} rooms | "
          f"{row.age_years}yr old | "
          f"{row.location_lbl}")
    print(f" Estimated Price → Rs {predictions[i]:>10,.0f}")
    print()

```

HOUSE PRICE PREDICTIONS

```

=====
House 1: 120 sqft | 2 rooms | 10yr old | Urban
Estimated Price → Rs      120,606

```

```
House 2: 250 sqft | 4 rooms | 19yr old | Suburban  
Estimated Price → Rs    152,340
```

```
House 3: 300 sqft | 3 rooms | 25yr old | Rural  
Estimated Price → Rs    106,398
```

```
House 4: 350 sqft | 6 rooms | 20yr old | Urban  
Estimated Price → Rs    230,020
```

Start coding or [generate](#) with AI.